

Topic 14: Sorting Algorithms

14.1 Bubble Sort

- Compare adjacent elements and swap if out of order -- largest bubbles to the end each pass
- Time: $O(n^2)$ worst and average, $O(n)$ best (already sorted with early-exit flag)
- Space: $O(1)$ in-place
- Stable sort -- preserves relative order of equal elements

```
void bubbleSort(int* a, int n) {
    for (int i = 0; i < n-1; i++) {
        bool swapped = false;
        for (int j = 0; j < n-1-i; j++)
            if (a[j] > a[j+1]) { swap(a[j], a[j+1]); swapped = true; }
        if (!swapped) break; // already sorted
    }
}
```

14.2 Selection Sort

- Find minimum in unsorted region, swap with first unsorted element -- repeat
- Time: $O(n^2)$ always -- never benefits from partially sorted input
- Minimises number of swaps -- $O(n)$ swaps total

```
void selectionSort(int* a, int n) {
    for (int i = 0; i < n-1; i++) {
        int minIdx = i;
        for (int j = i+1; j < n; j++)
            if (a[j] < a[minIdx]) minIdx = j;
        swap(a[i], a[minIdx]);
    }
}
```

14.3 Insertion Sort

- Build sorted portion one element at a time -- shift elements right to make room
- Time: $O(n^2)$ worst, $O(n)$ best (already sorted) -- excellent for small or nearly sorted arrays
- Stable, in-place, online (can sort as elements arrive)

```

void insertionSort(int* a, int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i], j = i-1;
        while (j >= 0 && a[j] > key) { a[j+1] = a[j]; j--; }
        a[j+1] = key;
    }
}

```

14.4 Merge Sort

- Divide array in half, recursively sort each half, merge sorted halves
- Time: $O(n \log n)$ always -- consistent performance
- Space: $O(n)$ extra -- requires auxiliary array during merge
- Stable sort -- preferred when stability matters

```

void merge(int* a, int l, int m, int r) {
    vector<int> left(a+l, a+m+1), right(a+m+1, a+r+1);
    int i=0, j=0, k=l;
    while (i<left.size() && j<right.size())
        a[k++] = (left[i]<=right[j]) ? left[i++] : right[j++];
    while (i<left.size()) a[k++] = left[i++];
    while (j<right.size()) a[k++] = right[j++];
}

```

14.5 Quick Sort

- Pick a pivot, partition array so elements < pivot are left, > pivot are right, recurse
- Time: $O(n \log n)$ average, $O(n^2)$ worst (bad pivot choice -- e.g., always smallest)
- Space: $O(\log n)$ stack space average
- In-place, cache-friendly -- fastest in practice for most inputs
- Pivot strategies: last element (simple), median-of-three (better), random (avoids worst case)

```
int partition(int* a, int l, int r) {
    int pivot = a[r], i = l-1;
    for (int j = l; j < r; j++)
        if (a[j] <= pivot) swap(a[++i], a[j]);
    swap(a[i+1], a[r]);
    return i+1;
}

void quickSort(int* a, int l, int r) {
    if (l < r) {
        int p = partition(a, l, r);
        quickSort(a, l, p-1);
        quickSort(a, p+1, r);
    }
}
```